

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



ViewModel and Debugging

Oleh:

Rika Fauliana Rahmi NIM. 2410817120017

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Rika Fauliana Rahmi
NIM : 2410817120017

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	6
B. Output Program	21
C. Pembahasan	23
SOAL 2.....	32
TAUTAN GIT	33

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 XML (Step Over).....	21
Gambar 2. Screenshot Hasil Jawaban Soal 1 XML (Step Into)	21
Gambar 3. Screenshot Hasil Jawaban Soal 1 XML (Step Out).....	22
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose (Step Over).....	22
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose (Step Into)	23
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose (Step Out).....	23

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt XML Soal 1	6
Tabel 2. Source Code MyApplication.kt XML Soal 1	7
Tabel 3. Source Code LegoViewModel.kt XML Soal 1	7
Tabel 4. Source Code LegoViewModelFactory.kt XML Soal 1	9
Tabel 5. Source Code ListFragment.kt XML Soal 1	9
Tabel 6. Source Code LegoAdapter.kt XML Soal 1	12
Tabel 7. Source Code LegoViewModel.kt Compose Soal 1	13
Tabel 8. Source Code MyApplication.kt Compose Soal 1	15
Tabel 9. Source Code MainActivity.kt Compose Soal 1	15
Tabel 10. Source Code AppScreen.kt Compose Soal 1	17

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- Install dan gunakan library Timber untuk logging event berikut:
- Log saat data item masuk ke dalam list
- Log saat tombol Detail dan tombol Explicit Intent ditekan
- Log data dari list yang dipilih ketika berpindah ke halaman Detail
- Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out.

A. Source Code

XML

MainActivity.kt

Tabel 1. Source Code MainActivity.kt XML Soal 1

1	package com.example.modul3xml
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import
	com.example.modul3xml.databinding.ActivityMainBinding
6	
7	class MainActivity : AppCompatActivity() {
8	private lateinit var binding: ActivityMainBinding
9	

10	override fun onCreate(savedInstanceState: Bundle?)
11	{
12	super.onCreate(savedInstanceState)
13	binding =
14	ActivityMainBinding.inflate(layoutInflater)
15	setContentView(binding.root)
16	}

MyApplication.kt

Tabel 2. Source Code MyApplication.kt XML Soal 1

1	package com.example.modul3xml
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	class MyApplication : Application() {
7	override fun onCreate() {
8	super.onCreate()
9	Timber.plant(Timber.DebugTree())
10	}
11	}

LegoViewModel.kt

Tabel 3. Source Code LegoViewModel.kt XML Soal 1

1	package com.example.modul3xml
2	
3	import androidx.lifecycle.ViewModel
4	import kotlinx.coroutines.flow.MutableStateFlow
5	import kotlinx.coroutines.flow.StateFlow
6	import kotlinx.coroutines.flow.asStateFlow
7	import timber.log.Timber
8	
9	class LegoViewModel(val namaAplikasi: String) :
10	ViewModel() {
11	private val _legoList =
12	MutableStateFlow<List<Lego>>(emptyList())
13	val legoList: StateFlow<List<Lego>> =
14	_legoList.asStateFlow()
15	private val _clickEvent =
16	MutableStateFlow<ClickEvent?>(null)

```

15     val clickEvent: StateFlow<ClickEvent?> =
16     _clickEvent.asStateFlow()
17
18     sealed class ClickEvent {
19         data class DetailClick(val lego: Lego) :
20         ClickEvent()
21         data class WebClick(val lego: Lego) :
22         ClickEvent()
23     }
24
25     init {
26         loadLegoData()
27     }
28
29     private fun loadLegoData() {
30         val list = listOf(
31             Lego(1, "Millennium Falcon", "2017", "Star
32             Wars", "7541 pcs", "Pesawat tempur legendaris Han Solo.",
33             R.drawable.lego1, "https://www.lego.com"),
34             Lego(2, "Hogwarts Castle", "2018", "Harry
35             Potter", "6020 pcs", "Kastil sekolah sihir Hogwarts.",
36             R.drawable.lego2, "https://www.lego.com"),
37             Lego(3, "Taj Mahal", "2021", "Architecture",
38             "2022 pcs", "Replika bangunan Taj Mahal.",
39             R.drawable.lego3, "https://www.lego.com"),
40             Lego(4, "Porsche 911", "2021", "Creator",
41             "1458 pcs", "Mobil klasik Porsche 911.",
42             R.drawable.lego4, "https://www.lego.com"),
43             Lego(5, "Central Perk", "2019", "Ideas",
44             "1070 pcs", "Kafe ikonik dari serial TV Friends.",
45             R.drawable.lego5, "https://www.lego.com")
46         )
47
48         Timber.d("[${namaAplikasi}] Data berhasil dimuat:
49         ${list.size} item")
50         list.forEach { lego ->
51             Timber.d("    → Item masuk: [${lego.id}]
52             ${lego.title} (${lego.year})")
53         }
54
55         _legoList.value = list
56     }
57
58     fun onDetailClick(lego: Lego) {
59         Timber.d("Tombol Detail ditekan →
60         ${lego.title}")
61         _clickEvent.value = ClickEvent.DetailClick(lego)
62     }

```


46	}
47	
48	fun onWebClick(lego: Lego) {
49	Timber.d("Tombol Explicit Intent (Situs) ditekan → \${lego.title} URL: \${lego.webUrl}")
50	_clickEvent.value = ClickEvent.WebClick(lego)
51	}
52	
53	fun resetClickEvent() {
54	_clickEvent.value = null
55	}
56	}

LegoViewModelFactory.kt

Tabel 4. Source Code LegoViewModelFactory.kt XML Soal 1

1	package com.example.modul3xml
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class LegoViewModelFactory(private val legoCollection: String) : ViewModelProvider.Factory {
7	
8	override fun <T : ViewModel> create(modelClass: Class<T>): T {
9	if (modelClass.isAssignableFrom(LegoViewModel::class.java))
10	{
11	@Suppress("UNCHECKED_CAST")
12	return LegoViewModel(legoCollection) as T
13	} throw IllegalArgumentException("ViewModel tidak dikenal")
14	}
15	}

ListFragment.kt

Tabel 5. Source Code ListFragment.kt XML Soal 1

1	package com.example.modul3xml
2	
3	import android.content.Intent
4	import android.os.Bundle
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup

```

8 import androidx.core.net.toUri
9 import androidx.fragment.app.Fragment
10 import androidx.lifecycle.ViewModelProvider
11 import androidx.lifecycle.lifecyleScope
12 import androidx.navigation.fragment.findNavController
13 import androidx.recyclerview.widget.LinearLayoutManager
14 import
15 com.example.modul3xml.databinding.FragmentListBinding
16 import kotlinx.coroutines.launch
17 import timber.log.Timber
18
19 class ListFragment : Fragment() {
20     private var _binding: FragmentListBinding? = null
21     private val binding get() = _binding!!
22
23     override fun onCreateView(
24         inflater: LayoutInflater, container: ViewGroup?,
25         savedInstanceState: Bundle?
26     ): View {
27         _binding = FragmentListBinding.inflate(inflater,
28 container, false)
29         return binding.root
30     }
31
32     override fun onViewCreated(view: View,
33 savedInstanceState: Bundle?) {
34         super.onViewCreated(view, savedInstanceState)
35
36         val factory = LegoViewModelFactory("Aplikasi
37 Lego Collection")
38         val viewModel = ViewModelProvider(this,
39 factory)[LegoViewModel::class.java]
40
41         binding.rvHighlight.layoutManager =
42 LinearLayoutManager(
43         requireContext(),
44         LinearLayoutManager.HORIZONTAL, false
45         )
46
47         // Collect StateFlow legoList
48         viewLifecycleOwner.lifecycleScope.launch {
49             viewModel.legoList.collect { list ->
50                 binding.rvHighlight.adapter =
51 HighlightAdapter(list)
52                 binding.rvLego.layoutManager =
53 LinearLayoutManager(requireContext())

```

```

46             binding.rvLego.adapter = LegoAdapter(
47                 listData = list,
48                 onDetailClick = { lego ->
viewModel.onDetailClick(lego) },
49                 onWebClick = { lego ->
viewModel.onWebClick(lego) }
50             )
51         }
52     }
53
54     // Collect StateFlow clickEvent
55     viewLifecycleOwner.lifecycleScope.launch {
56         viewModel.clickEvent.collect { event ->
57             when (event) {
58                 is
LegoViewModel.ClickEvent.DetailClick -> {
59                     Timber.d("Berpindah ke Detail →
ID: ${event.lego.id} | Judul: ${event.lego.title} | Tema:
${event.lego.theme}")
60                     val bundle = Bundle()
61
62                     bundle.putParcelable("data_lego", event.lego)
63
64                     findNavController().navigate(R.id.action_list_to_detail
, bundle)
65
66                     viewModel.resetClickEvent()
67                 }
68                 is
LegoViewModel.ClickEvent.WebClick -> {
69                     val intent =
Intent(Intent.ACTION_VIEW, event.lego.webUrl.toUri())
70                     startActivity(intent)
71                     viewModel.resetClickEvent()
72                 }
73             }
74
75             binding.btnLanguage.setOnClickListener {
76                 findNavController().navigate(R.id.action_listFragment_t
o_languageFragment)
77             }
78         }
79
80         override fun onDestroyView() {

```

81	super.onDestroyView()
82	_binding = null
83	}
84	}

LegoAdapter.kt

Tabel 6. Source Code LegoAdapter.kt XML Soal 1

1	package com.example.modul3xml
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import
	com.example.modul3xml.databinding.ItemLegoBinding
7	
8	class LegoAdapter(
9	private val listData: List<Lego>,
10	private val onDetailClick: (Lego) -> Unit,
11	private val onWebClick: (Lego) -> Unit
12	) : RecyclerView.Adapter<LegoAdapter.LegoViewHolder>() {
13	
14	class LegoViewHolder(val binding: ItemLegoBinding) :
	RecyclerView.ViewHolder(binding.root)
15	
16	override fun onCreateViewHolder(parent: ViewGroup,
	viewType: Int): LegoViewHolder {
17	val view =
	ItemLegoBinding.inflate(LayoutInflater.from(parent.context), parent, false)
18	return LegoViewHolder(view)
19	}
20	
21	override fun onBindViewHolder(holder: LegoViewHolder, position: Int) {
22	val item = listData[position]
23	
24	holder.binding.tvTitle.text = item.title
25	holder.binding.tvYear.text = item.year
26	holder.binding.tvTheme.text = item.theme
27	holder.binding.tvDescription.text =
	item.description
28	
	holder.binding.ivLego.setImageResource(item.imageRes)
29	
30	holder.binding.btnWeb.setOnClickListener {
31	onWebClick(item)

32	}
33	
34	holder.binding.btnDetail.setOnClickListener {
35	onDetailClick(item)
36	}
37	}
38	
39	override fun getItemCount(): Int = listData.size
40	}

Jetpack Compose

LegoViewModel.kt

Tabel 7. Source Code LegoViewModel.kt Compose Soal 1

1	package com.example.modul3compose
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import kotlinx.coroutines.flow.MutableStateFlow
6	import kotlinx.coroutines.flow.StateFlow
7	import kotlinx.coroutines.flow.asStateFlow
8	import timber.log.Timber
9	
10	class LegoViewModel(val legoCollection: String) : ViewModel() {
11	
12	// StateFlow data list
13	private val _legoList = MutableStateFlow<List<Lego>>(emptyList())
14	val legoList: StateFlow<List<Lego>> = _legoList.asStateFlow()
15	
16	// StateFlow event klik
17	private val _clickEvent = MutableStateFlow<ClickEvent?>(null)
18	val clickEvent: StateFlow<ClickEvent?> = _clickEvent.asStateFlow()
19	
20	sealed class ClickEvent {
21	data class DetailClick(val lego: Lego) : ClickEvent()
22	data class WebClick(val lego: Lego) : ClickEvent()
23	}
24	
25	init {

```

26         loadLegoData()
27     }
28
29     private fun loadLegoData() {
30         val list = getDummyLegoList()
31
32         Timber.d("[${legoCollection}      Data      dimuat:
33         ${list.size} item")
34         list.forEach { lego ->
35             Timber.d("      → Item      masuk:      [${lego.id}]
36             ${lego.title}")
37         }
38         _legoList.value = list
39     }
40
41     fun onDetailClick(lego: Lego) {
42         Timber.d("Tombol Detail ditekan → ${lego.title}")
43         _clickEvent.value = ClickEvent.DetailClick(lego)
44     }
45
46     fun onWebClick(lego: Lego) {
47         Timber.d("Tombol Web/Explicit Intent ditekan →
48         ${lego.title}")
49         _clickEvent.value = ClickEvent.WebClick(lego)
50     }
51
52     fun resetClickEvent() {
53         _clickEvent.value = null
54     }
55
56     class LegoViewModelFactory(private val namaAplikasi:
57     String) : ViewModelProvider.Factory {
58         override fun <T : ViewModel> create(modelClass:
59         Class<T>): T {
60             if
61             (modelClass.isAssignableFrom(LegoViewModel::class.java))
62             {
63                 @Suppress("UNCHECKED_CAST")
64                 return LegoViewModel(namaAplikasi) as T
65             }
66             throw IllegalArgumentException("ViewModel tidak
67             dikenal")
68         }
69     }

```

MyApplication.kt

Tabel 8. Source Code MyApplication.kt Compose Soal 1

1	package com.example.modul3compose
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	class MyApplication : Application() {
7	override fun onCreate() {
8	super.onCreate()
9	Timber.plant(Timber.DebugTree())
10	}
11	}

MainActivity.kt

Tabel 9. Source Code MainActivity.kt Compose Soal 1

1	package com.example.modul3compose
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import androidx.activity.ComponentActivity
7	import androidx.activity.compose.setContent
8	import androidx.compose.material3.MaterialTheme
9	import androidx.compose.material3.Surface
10	import androidx.compose.runtime.LaunchedEffect
11	import androidx.compose.runtime.collectAsState
12	import androidx.compose.runtime.getValue
13	import androidx.compose.ui.Modifier
14	import androidx.compose.foundation.layout.fillMaxSize
15	import androidx.lifecycle.viewmodel.compose.viewModel
16	import androidx.navigation.compose.NavHost
17	import androidx.navigation.compose.composable
18	import
	androidx.navigation.compose.rememberNavController
19	import timber.log.Timber
20	
21	class MainActivity : ComponentActivity() {
22	override fun onCreate(savedInstanceState: Bundle?)
	{
23	super.onCreate(savedInstanceState)
24	
25	setContent {
26	MaterialTheme {

```

27         Surface(modifier =
Modifier.fillMaxSize(), color =
MaterialTheme.colorScheme.background) {
28
29             val navController =
rememberNavController()
30
31             val factory =
LegoViewModelFactory("Aplikasi Lego Compose")
32             val viewModel: LegoViewModel =
viewModel(factory = factory)
33
34             // Collect StateFlow
35             val legoList by
viewModel.legoList.collectAsState()
36             val clickEvent by
viewModel.clickEvent.collectAsState()
37
38             // Handle event klik from ViewModel
39             LaunchedEffect(clickEvent) {
40                 when (val event = clickEvent) {
41                     is
LegoViewModel.ClickEvent.DetailClick -> {
42                         Timber.d("Berpindah ke
Detail → ID: ${event.lego.id} | Judul:
${event.lego.title}")
43                         navController.navigate("detail_screen/${event.lego.id}"
)
44                         viewModel.resetClickEvent()
45                     }
46                     is
LegoViewModel.ClickEvent.WebClick -> {
47                         startActivity(Intent(Intent.ACTION_VIEW,
Uri.parse(event.lego.webUrl)))
48                         viewModel.resetClickEvent()
49                     }
50                     null -> {}
51                 }
52             }
53
54             NavHost(navController =
navController, startDestination = "list_screen") {
55                 composable("list_screen") {

```


56	ListScreen(
57	legoList = legoList,
58	onDetailClick = { lego
59	-> viewModel.onDetailClick(lego) },
59	onWebClick = { lego ->
60	viewModel.onWebClick(lego) },
60	onNavigateToLanguage =
61	{ navController.navigate("language_screen") }
61	)
62	}
63	
64	composable("detail_screen/{legoId}") { backStackEntry -
65	>
65	val id =
66	backStackEntry.arguments?.getString("legoId")?.toIntOrNull()
66	val selectedLego =
67	legoList.find { it.id == id }
67	if (selectedLego != null) {
68	DetailScreen(lego =
69	selectedLego)
69	}
70	}
71	
72	composable("language_screen") {
73	LanguageScreen()
74	}
75	}
76	}
77	}
78	}
79	}
80	}

AppScreen.kt

Tabel 10. Source Code AppScreen.kt Compose Soal 1

1	package com.example.modul3compose
2	
3	import android.app.Activity
4	import android.content.Context
5	import androidx.compose.foundation.Image
6	import androidx.compose.foundation.layout.*
7	import androidx.compose.foundation.lazy.LazyColumn
8	import androidx.compose.foundation.lazy.LazyRow
9	import androidx.compose.foundation.lazy.items

```

10 import
   androidx.compose.foundation.shape.RoundedCornerShape
11 import androidx.compose.material.icons.Icons
12 import androidx.compose.material.icons.filled.Language
13 import androidx.compose.material3.*
14 import androidx.compose.runtime.Composable
15 import androidx.compose.ui.Alignment
16 import androidx.compose.ui.Modifier
17 import androidx.compose.ui.draw.clip
18 import androidx.compose.ui.layout.ContentScale
19 import androidx.compose.ui.platform.LocalContext
20 import androidx.compose.ui.res.painterResource
21 import androidx.compose.ui.res.stringResource
22 import androidx.compose.ui.text.font.FontWeight
23 import androidx.compose.ui.unit.dp
24 import androidx.compose.ui.unit.sp
25 import java.util.Locale
26
27 @OptIn(ExperimentalMaterial3Api::class)
28 @Composable
29 fun ListScreen(
30     legoList: List<Lego>,
31     onDetailClick: (Lego) -> Unit,
32     onWebClick: (Lego) -> Unit,
33     onNavigateToLanguage: () -> Unit
34 ) {
35     Scaffold(
36         topBar = {
37             TopAppBar(
38                 title = {
39                     Text(stringResource(R.string.app_name), fontWeight =
40                         FontWeight.Bold) },
41                 actions = {
42                     IconButton(onClick =
43                         onNavigateToLanguage) {
44                         Icon(Icons.Default.Language,
45                             contentDescription = "Lang")
46                     }
47                 }
48             )
49         }
50     ) { padding ->
51         LazyColumn(
52             modifier =
53                 Modifier.padding(padding).fillMaxSize(),
54             contentPadding = PaddingValues(16.dp),

```

```

50         verticalArrangement =
Arrangement.spacedBy(16.dp)
51     ) {
52         item {
53
54             Text(stringResource(R.string.title_highlight),
fontWeight = FontWeight.Bold, fontSize = 18.sp)
55             LazyRow(modifier = Modifier.padding(top
= 8.dp), horizontalArrangement =
Arrangement.spacedBy(12.dp)) {
56                 items(legoList) { lego ->
57                     Image(
58                         painter =
painterResource(lego.imageRes), contentDescription =
null,
59                         modifier =
Modifier.size(260.dp,
150.dp).clip(RoundedCornerShape(16.dp)),
60                         contentScale =
ContentScale.Crop
61                     )
62                 }
63             Spacer(modifier =
Modifier.height(16.dp))
64
65             Text(stringResource(R.string.title_all_items),
fontWeight = FontWeight.Bold, fontSize = 18.sp)
66         }
67         items(legoList) { lego ->
68             LegoItem(
69                 lego = lego,
70                 onWebClick = { onWebClick(lego) },
71                 onDetailClick = {
onDetailClick(lego) }
72             )
73         }
74     }
75 }
76
77
78 @Composable
79 fun DetailScreen(lego: Lego) {
80     Column(modifier =
Modifier.fillMaxSize().padding(16.dp),
horizontalAlignment = Alignment.CenterHorizontally) {

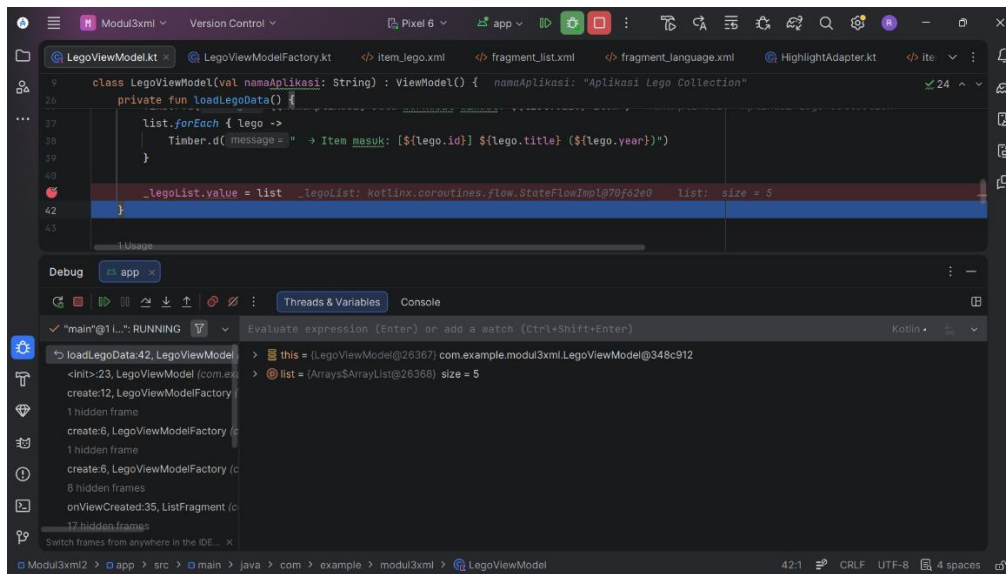
```

```

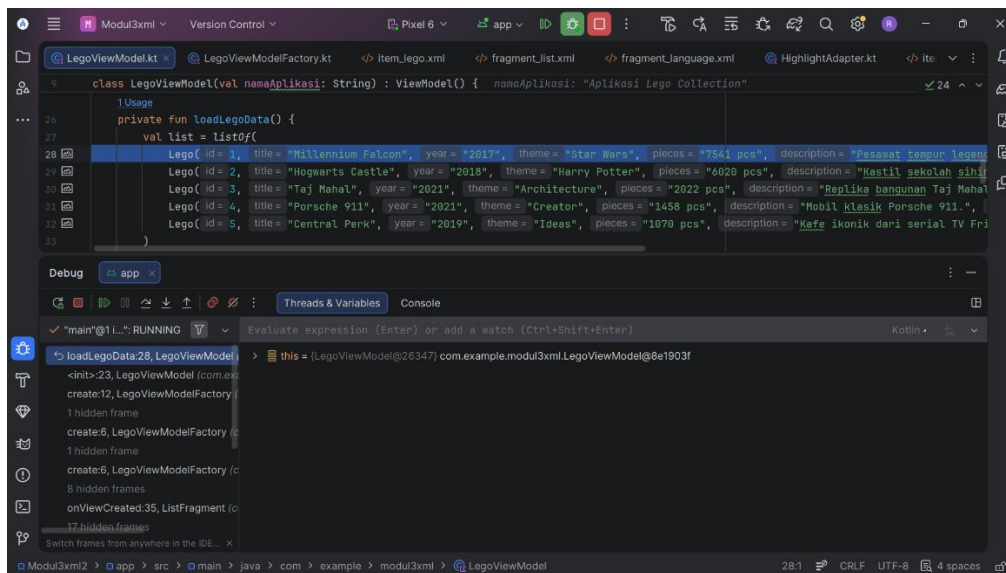
81         Image(painter = painterResource(lego.imageRes),
contentDescription = null, modifier =
Modifier.size(200.dp).clip(RoundedCornerShape(16.dp)),
contentScale = ContentScale.Crop)
82         Spacer(modifier = Modifier.height(16.dp))
83         Text(text = lego.title, fontSize = 24.sp,
fontWeight = FontWeight.Bold)
84         Text(text =
"${stringResource(R.string.label_year)} ${lego.year}",
fontSize = 16.sp)
85         Text(text =
"${stringResource(R.string.label_theme)}
${lego.theme}", fontSize = 16.sp)
86         Spacer(modifier = Modifier.height(16.dp))
87         Text(text = lego.description, fontSize = 14.sp)
88     }
89 }
90
91 @Composable
92 fun LanguageScreen() {
93     val context = LocalContext.current
94     Column(modifier = Modifier.fillMaxSize(),
verticalArrangement = Arrangement.Center,
horizontalAlignment = Alignment.CenterHorizontally) {
95         Text(stringResource(R.string.title_language),
fontSize = 22.sp, fontWeight = FontWeight.Bold)
96         Spacer(modifier = Modifier.height(24.dp))
97         Button(onClick = { updateLocale(context, "in")
}) { Text("Bahasa Indonesia") }
98         Spacer(modifier = Modifier.height(8.dp))
99         Button(onClick = { updateLocale(context, "en")
}) { Text("English") }
100     }
101 }
102
103 fun updateLocale(context: Context, lang: String) {
104     val locale = Locale(lang)
105     Locale.setDefault(locale)
106     val config = context.resources.configuration
107     config.setLocale(locale)
108     context.resources.updateConfiguration(config,
context.resources.displayMetrics)
109     (context as? Activity)?.recreate() // Refresh
activity
110 }

```

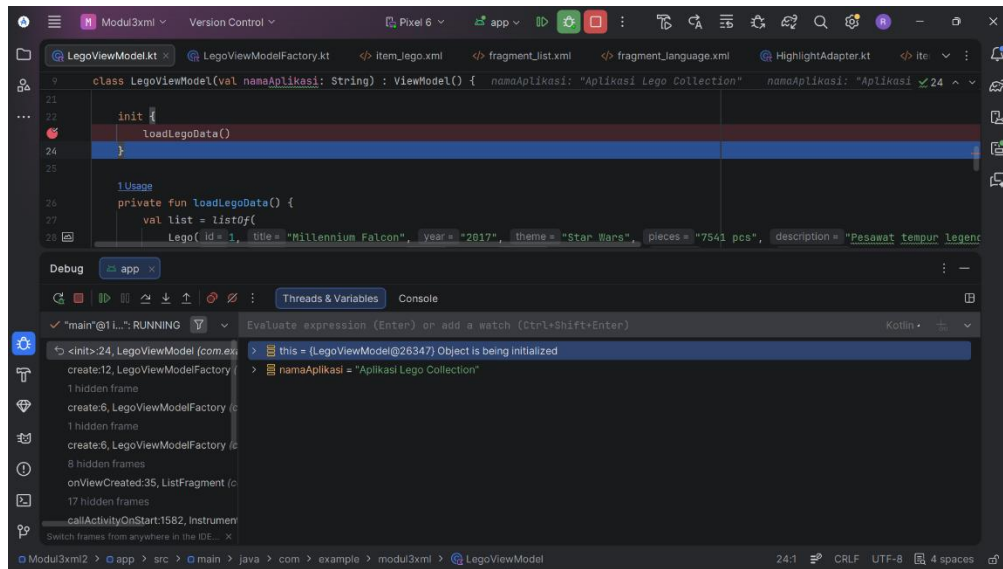
B. Output Program XML



Gambar 1. Screenshot Hasil Jawaban Soal 1 XML (Step Over)

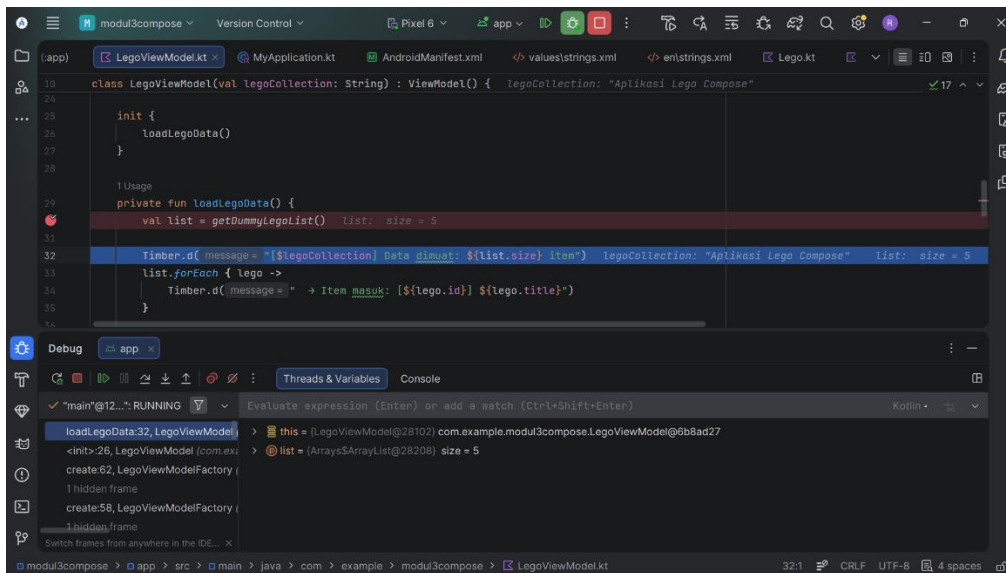


Gambar 2. Screenshot Hasil Jawaban Soal 1 XML (Step Into)

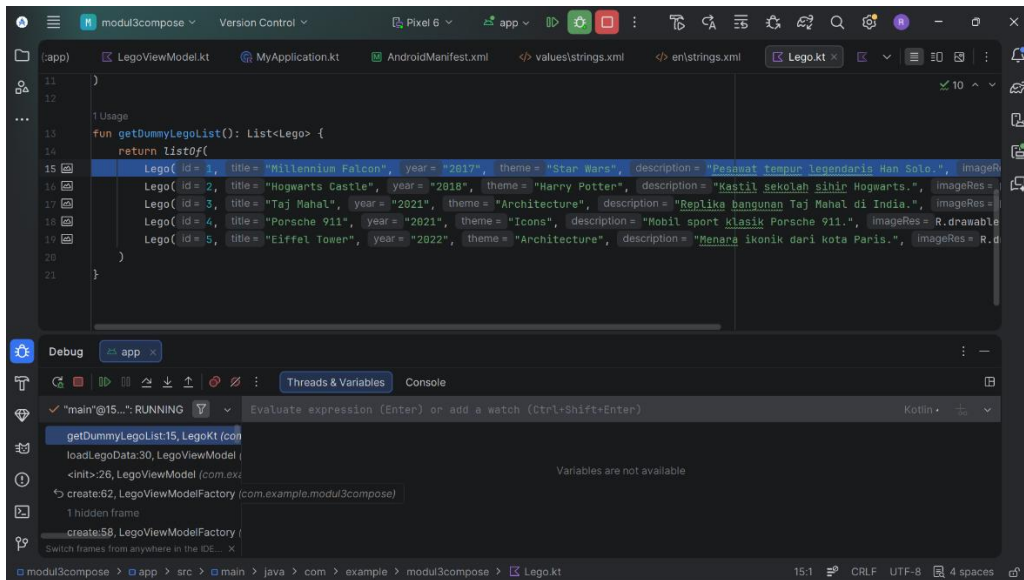


Gambar 3. Screenshot Hasil Jawaban Soal 1 XML (Step Out)

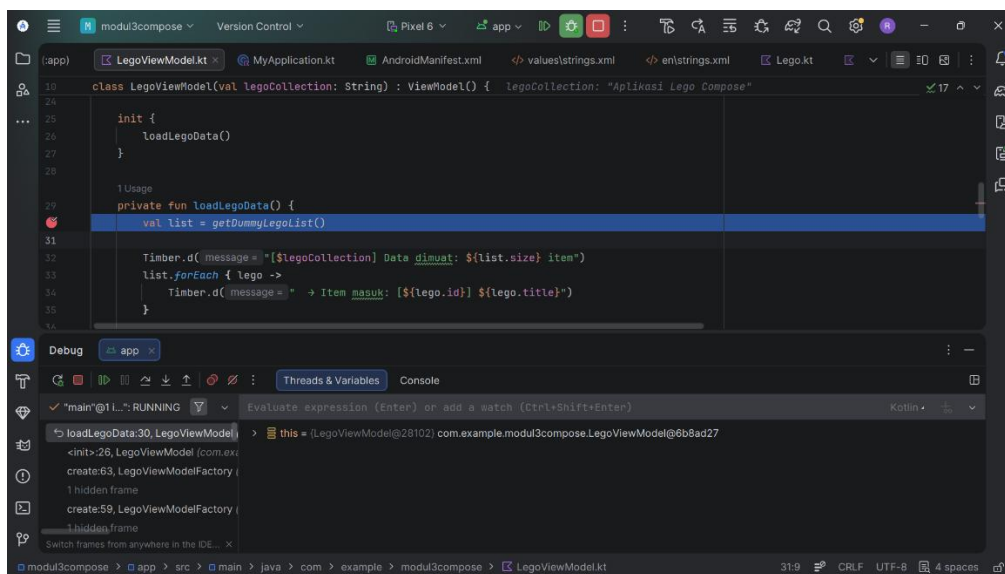
Jetpack Compose



Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose (Step Over)



Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose (Step Into)



Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose (Step Out)

C. Pembahasan

XML

MainActivity.kt

Pada file MainActivity.kt, logika aplikasi mulai berjalan di dalam fungsi 'onCreate()'. Sebelum masuk ke dalam fungsi tersebut, program mendeklarasikan sebuah variabel bernama binding dari class ActivityMainBinding untuk menghubungkan komponen UI, di

mana inisialisasinya ditunda terlebih dahulu menggunakan `lateinit`. Saat fungsi `'onCreate()'` dieksekusi, langkah pertama yang dilakukan adalah memanggil fungsi `'onCreate()'` dari kelas induknya untuk memastikan siklus hidup dan status dasar aktivitas dibangun dengan benar oleh sistem.

Lalu, dilakukan proses inisialisasi pada variabel binding dengan memanggil fungsi `'inflate()'` yang membawa `layoutInflater` untuk merender dan menghubungkan class Kotlin ini dengan seluruh komponen desain XML yang telah dibuat. Setelah proses inisialisasi tersebut selesai, program memanggil fungsi `'setContentView()'` dengan menyematkan akar tata letaknya, yaitu `binding.root`, untuk secara resmi menetapkan dan menampilkan struktur antarmuka tersebut ke layar perangkat pengguna.

MyApplication.kt

Pada file `MyApplication.kt`, logika inisialisasi aplikasi mulai berjalan di dalam fungsi `'onCreate()'`. Fungsi `'onCreate()'` dari superclass dipanggil terlebih dahulu untuk memastikan setup dasar dari class `Application` dibangun dengan benar oleh sistem. Lalu, program memanggil fungsi `'plant()'` dari library `Timber` dengan menyematkan `Timber.DebugTree()` sebagai parameternya. Proses ini dilakukan untuk menghubungkan dan mengaktifkan sistem logging, sehingga memastikan bahwa aplikasi dapat mencetak log secara otomatis untuk mempermudah pemantauan selama program berjalan pada mode debug.

LegoViewModel.kt

Pada file `LegoViewModel.kt`, logika pengelolaan data dimulai di dalam class `LegoViewModel` yang merupakan turunan dari class `ViewModel`. Di awal class, dilakukan proses inisialisasi variabel state menggunakan `MutableStateFlow` untuk menyimpan daftar objek `Lego` dan event interaksi. Variabel `_legoList` dan `_clickEvent` yang bersifat `private` ini kemudian dihubungkan menjadi `StateFlow` yang bersifat `read-only` dengan memanggil fungsi `'asStateFlow()'`. Program juga mendefinisikan sebuah sealed class bernama `ClickEvent` yang memuat data class untuk membedakan antara aksi `DetailClick` dan `WebClick`. Setelah proses inisialisasi selesai, program memanggil fungsi `'loadLegoData()'` di dalam blok `init` agar proses pengambilan data langsung dijalankan secara otomatis saat class ini dibentuk.

Di dalam fungsi 'loadLegoData()', kumpulan data simulasi yang memuat id, nama, tahun, tema, jumlah komponen, deskripsi, gambar, dan URL dari berbagai set Lego dimasukkan ke dalam sebuah list. Setelah list tersebut berhasil disusun, program memanggil fungsi 'd()' dari library Timber untuk mencatat total item dan menelusuri keseluruhan data satu per satu agar tercetak ke dalam log sistem. Setelah pencatatan selesai, program secara langsung memperbarui state dengan memasukkan list tersebut ke dalam `_legoList.value` sehingga data siap untuk ditampilkan oleh komponen UI.

Aksi penanganan interaksinya terjadi di dalam fungsi 'onDetailClick()' dan 'onWebClick()'. Ketika pengguna memilih sebuah item Lego untuk melihat rinciannya, blok kode di dalam fungsi 'onDetailClick()' akan menangkap objek tersebut, memanggil fungsi 'd()' untuk mencetak log, dan langsung memperbarui nilai `_clickEvent.value` dengan event DetailClick. Sementara itu, jika pengguna menekan tombol tautan situs, program memanggil fungsi 'onWebClick()' yang juga akan mencatat log informasi URL menggunakan fungsi 'd()' dan mengubah state menjadi WebClick. Terakhir, program menyediakan fungsi 'resetClickEvent()' untuk mengubah status `_clickEvent.value` kembali menjadi null, yang berfungsi mereset state agar navigasi atau aksi klik tidak terus terpicu secara berulang oleh sistem.

LegoViewModelFactory.kt

Pada file `LegoViewModelFactory.kt`, logika pembuatannya diatur di dalam sebuah custom class bernama `LegoViewModelFactory` yang mengimplementasikan interface dari `ViewModelProvider.Factory`. Class ini dibuat secara khusus untuk memfasilitasi pembuatan instance dari `ViewModel` yang membutuhkan parameter tambahan saat inisialisasi, yang dalam hal ini adalah sebuah data string bernama `legoCollection`. Alur eksekusi utamanya berjalan di dalam fungsi 'create()' yang di override dari interface bawaannya untuk mengambil alih cara sistem Android mencetak objek `ViewModel` tersebut.

Di dalam fungsi 'create()', program menjalankan struktur percabangan if untuk mengecek apakah model class yang diminta oleh sistem memiliki tipe yang sesuai dengan class `LegoViewModel` dengan memanggil fungsi 'isAssignableFrom()'. Jika kondisi tersebut terpenuhi, program akan membentuk instance baru dari `LegoViewModel` yang diisi dengan nilai `legoCollection`, lalu mengembalikannya sebagai tipe generik T dengan menyematkan

anotasi `@Suppress` untuk mengabaikan peringatan sistem agar proses casting tipe data dianggap aman. Sebaliknya, jika tipe class yang dimasukkan ternyata tidak sesuai, program akan menghentikan eksekusi dan melempar error `IllegalArgumentException` untuk memastikan bahwa aplikasi tidak berjalan dengan `ViewModel` yang salah.

ListFragment.kt

Pada file `ListFragment.kt`, proses pembentukan tampilan mulai berjalan di dalam fungsi `'onCreateView()'`, di mana program memanggil fungsi `'inflate()'` untuk merender desain XML menjadi view nyata dan mengembalikannya melalui `binding.root`. Setelah tampilan dimuat, eksekusi utama berlanjut di dalam fungsi `'onViewCreated()'`. Di sini, program mengambil instance dari `LegoViewModel` dengan memanggil fungsi `'ViewModelProvider()'`, lalu mengatur orientasi daftar sorotan dengan memanggil fungsi `'LinearLayoutManager()'`.

Untuk menampilkan data, program memanggil fungsi `'launch()'` dan `'collect()'` guna memantau state `legoList` secara asynchronous. Saat data list diterima, program langsung menyusun daftar dengan memanggil konstruktor `'HighlightAdapter()'` dan `'LegoAdapter()'`, sekaligus menyematkan fungsi lambda yang akan memanggil fungsi `'onDetailClick()'` dan `'onWebClick()'` ke dalam view model ketika pengguna menekan sebuah item.

Selanjutnya, program kembali memanggil fungsi `'launch()'` dan `'collect()'` untuk mendengarkan instruksi dari state `clickEvent`. Jika event bernilai `DetailClick`, program memuat data ke dalam pemanggilan `'Bundle()'` lalu memanggil fungsi `'navigate()'` untuk berpindah halaman. Jika bernilai `WebClick`, program memanggil fungsi `'Intent()'` dan `'startActivity()'` untuk membuka tautan pada browser. Setelah salah satu aksi tersebut dijalankan, program selalu memanggil fungsi `'resetClickEvent()'` untuk mereset status. Program juga menempelkan aksi pada tombol bahasa dengan memanggil fungsi `'setOnClickListener()'` untuk memicu pemanggilan fungsi `'navigate()'`. Terakhir, untuk mencegah kebocoran memori, program memanggil fungsi `'onDestroyView()'` dan mengubah nilai binding menjadi null saat fragment dihancurkan oleh sistem.

LegoAdapter.kt

Pada file `LegoAdapter.kt`, logika pengelolaan daftar antarmuka diatur di dalam class `LegoAdapter` yang merupakan turunan dari class `RecyclerView.Adapter`. Di awal class, program menerima parameter utama berupa list data dari objek `Lego` beserta dua fungsi lambda untuk menangani aksi interaksi pengguna. Di dalam class tersebut, program mendefinisikan sebuah inner class bernama `LegoViewHolder` yang diturunkan dari `RecyclerView.ViewHolder` dan bertugas untuk menyimpan serta mengelola binding dari setiap komponen view secara individual.

Proses pembuatan kerangka tampilan mulai berjalan di dalam fungsi `'onCreateViewHolder()'`. Di dalam fungsi tersebut, program memanggil fungsi `'inflate()'` menggunakan objek `layoutInflater` untuk merender dan mengubah rancangan desain `XML ItemLegoBinding` menjadi view nyata, lalu mengembalikannya sebagai sebuah instance dari `LegoViewHolder` yang siap digunakan. Sementara itu, untuk menentukan total elemen yang akan ditampilkan ke layar, program memanggil fungsi `'getItemCount()'` yang secara langsung mengembalikan nilai berupa ukuran dari list data yang dimiliki.

Aksi pengikatan data dengan komponen antarmuka terjadi secara spesifik di dalam fungsi `'onBindViewHolder()'`. Saat fungsi ini dieksekusi oleh sistem, langkah pertama yang dilakukan adalah mengambil objek data spesifik berdasarkan urutan posisinya di dalam list. Selanjutnya, program memuat dan menempelkan teks seperti judul, tahun, tema, dan deskripsi ke antarmuka, lalu memanggil fungsi `'setImageResource()'` untuk menyematkan gambar ke dalam komponen UI yang terhubung pada binding. Terakhir, program menempelkan aksi deteksi klik pada tombol web dan tombol detail dengan memanggil fungsi `'setOnClickListener()'`, di mana kedua listener ini memastikan bahwa setiap interaksi klik akan langsung memanggil fungsi lambda yang telah disediakan sambil membawa data item tersebut secara otomatis.

Jetpack Compose

LegoViewModel.kt

Logika pengelolaan data dimulai di dalam class `LegoViewModel` yang merupakan turunan dari class `ViewModel`. Di awal class, program melakukan inisialisasi variabel state dengan memanggil fungsi `'MutableStateFlow()'` untuk menyimpan daftar objek `Lego` dan event interaksi pengguna. Variabel `_legoList` dan `_clickEvent` tersebut kemudian diubah

menjadi state yang bersifat read-only dengan memanggil fungsi 'asStateFlow()'. Setelah mendefinisikan sealed class untuk mengelompokkan jenis klik, program memanggil fungsi 'loadLegoData()' di dalam blok init agar proses pengambilan data langsung dijalankan secara otomatis saat class ini dibentuk.

Di dalam fungsi 'loadLegoData()', program memanggil fungsi 'getDummyLegoList()' untuk mengambil kumpulan data simulasi. Setelah data diperoleh, program memanggil fungsi 'd()' dari library Timber untuk mencetak informasi jumlah dan detail item ke dalam log sistem, lalu memperbarui state antarmuka secara langsung dengan memasukkan data tersebut ke dalam _legoList.value.

Aksi penanganan interaksinya diatur melalui pemanggilan fungsi 'onDetailClick()' dan 'onWebClick()'. Ketika pengguna memicu salah satu dari fungsi tersebut, program akan memanggil fungsi 'd()' untuk mencetak log aktivitas, lalu memperbarui nilai _clickEvent.value dengan event DetailClick atau WebClick. Selain itu, program juga menyediakan fungsi 'resetClickEvent()' untuk mengubah status event kembali menjadi null guna mereset state agar navigasi tidak terpicu secara berulang.

Sementara itu, logika pembuatan view model diatur dalam class LegoViewModelFactory yang mengimplementasikan interface ViewModelProvider.Factory. Alur utamanya berjalan di dalam fungsi 'create()' yang di-override untuk mengambil alih cara sistem mencetak objek ViewModel. Di dalam fungsi 'create()', program mengecek kecocokan tipe class dengan memanggil fungsi 'isAssignableFrom()'. Jika kondisinya sesuai, program akan membentuk dan mengembalikan instance dari LegoViewModel yang diisi dengan data string bawaannya, atau melempar error IllegalArgumentException jika tipe ViewModel tersebut tidak dikenali oleh sistem.

MyApplication.kt

Pada file MyApplication.kt, logika inisialisasi aplikasi mulai berjalan di dalam fungsi 'onCreate()'. Pertama-tama, fungsi 'onCreate()' dari superclass dipanggil terlebih dahulu untuk memastikan setup dasar dari class Application dibangun dengan benar oleh sistem. Setelah itu, program memanggil fungsi 'plant()' dari library Timber dengan menyematkan pemanggilan fungsi 'DebugTree()' sebagai parameteranya. Proses ini dilakukan untuk menghubungkan dan mengaktifkan sistem logging, sehingga memastikan bahwa aplikasi

dapat mencetak log secara otomatis guna mempermudah pemantauan selama program berjalan pada mode debug.

MainActivity.kt

Pada file MainActivity.kt, logika utama mulai berjalan di dalam fungsi 'onCreate()'. Setelah memanggil fungsi 'onCreate()' dari superclass, program memanggil fungsi 'setContent()' dan 'Surface()' dengan menyematkan pengubah 'fillMaxSize()' untuk menyusun kerangka antarmuka dasar berbasis Jetpack Compose.

Selanjutnya, program menyiapkan navigasi dengan memanggil fungsi 'rememberNavController()', lalu mengambil instance dari LegoViewModel dengan memanggil fungsi 'viewModel()'. Agar tampilan dapat memperbarui diri secara otomatis, program memantau aliran data pada variabel legoList dan clickEvent dengan memanggil fungsi 'collectAsState()'.

Aksi interaksi pengguna dipantau secara reaktif dengan memanggil fungsi 'LaunchedEffect()'. Jika event bernilai DetailClick, program langsung memanggil fungsi 'navigate()' untuk berpindah ke rute detail. Namun, jika bernilai WebClick, program memanggil fungsi 'Intent()' dan 'startActivity()' untuk membuka tautan eksternal pada browser. Setelah salah satu aksi tersebut dieksekusi, program selalu memanggil fungsi 'resetClickEvent()' untuk mereset status.

Alur perpindahan halamannya diatur dengan memanggil fungsi 'NavHost()'. Di dalam blok tersebut, program mendaftarkan tiga rute menggunakan pemanggilan fungsi 'composable()'. Rute list_screen secara langsung memanggil komponen 'ListScreen()', rute detail_screen memproses parameter ID menggunakan pemanggilan fungsi 'toIntOrNull()' dan 'find()' sebelum memanggil 'DetailScreen()', serta rute language_screen yang bertugas menampilkan halamannya dengan memanggil komponen 'LanguageScreen()'.

AppScreens.kt

Pada antarmuka ListScreen, proses pembentukan tampilan mulai berjalan dengan memanggil fungsi 'Scaffold()' untuk menyusun struktur dasar halaman. Di bagian atas layar, program memanggil fungsi 'TopAppBar()' yang memuat teks judul serta sebuah tombol ikon yang akan memicu event navigasi saat ditekan. Di dalam kerangka utamanya, program

menyusun konten yang dapat di-scroll secara vertikal dengan memanggil fungsi 'LazyColumn()'. Di bagian atas daftar tersebut, program menyematkan daftar sorotan horizontal dengan memanggil fungsi 'LazyRow()', di mana setiap elemennya dirender dengan memanggil fungsi 'Image()'. Setelah itu, program memanggil fungsi 'items()' untuk memuat seluruh sisa data ke dalam daftar vertikal dengan memanggil komponen kustom 'LegoItem()', sekaligus menempelkan lambda untuk menangani aksi klik pengguna.

Untuk antarmuka DetailScreen, tata letaknya disusun memanjang ke bawah dengan memanggil fungsi 'Column()'. Di dalam blok tersebut, program memanggil fungsi 'Image()' untuk menampilkan visualisasi Lego dengan sudut yang melengkung, lalu menyisipkan jarak antar komponen dengan memanggil fungsi 'Spacer()'. Selanjutnya, program merender keseluruhan informasi detail dengan memanggil fungsi 'Text()' secara berulang untuk menampilkan judul utama, tahun rilis, tema, hingga deskripsi lengkap dari item Lego yang sedang dilihat oleh pengguna.

Pada bagian pengaturan bahasa, komponen LanguageScreen dibangun dengan mengambil referensi context terlebih dahulu, lalu memanggil fungsi 'Column()' untuk memosisikan isinya ke tengah layar. Program kemudian menyusun dua buah tombol dengan memanggil fungsi 'Button()', di mana setiap kliknya akan langsung memicu pemanggilan fungsi 'updateLocale()'. Di dalam fungsi 'updateLocale()', program menerapkan bahasa pilihan pengguna dengan memanggil fungsi 'setDefault()' dan 'setLocale()' pada objek Locale. Setelah itu, konfigurasi sistem diperbarui dengan memanggil fungsi 'updateConfiguration()', sebelum akhirnya program merestart instance dari class Activity dengan memanggil fungsi 'recreate()' agar perubahan bahasa tersebut dapat langsung diterapkan secara visual.

Debugger:

Debugger adalah alat yang disediakan oleh Android Studio untuk membantu developer menemukan dan memperbaiki bug dalam aplikasi. Cara kerjanya adalah dengan menghentikan eksekusi program di titik tertentu yang disebut breakpoint, sehingga developer bisa mengamati kondisi aplikasi secara langsung.

Fitur Step Into, Step Over, dan Step Out:

1. Step Over digunakan untuk menjalankan baris kode saat ini dan melompat ke baris berikutnya, tanpa masuk ke dalam fungsi yang dipanggil di baris tersebut. Misalnya jika ada baris `_legoList.value = list`, Step Over akan menjalankan baris itu dan langsung pindah ke baris berikutnya.
2. Step Into digunakan untuk masuk ke dalam fungsi yang dipanggil di baris saat ini. Misalnya jika baris saat ini adalah `loadLegoData()`, menekan Step Into akan memindahkan kursor ke baris pertama di dalam fungsi `loadLegoData()`. Fitur ini berguna ketika ingin mengetahui secara detail apa yang terjadi di dalam suatu fungsi, terutama saat mencurigai bug ada di dalamnya.
3. Step Out digunakan untuk keluar dari fungsi yang sedang dieksekusi dan kembali ke fungsi yang memanggilnya. Misalnya jika kamu sedang berada di dalam `loadLegoData()`, menekan Step Out akan menyelesaikan eksekusi fungsi tersebut dan memindahkan kursor kembali ke baris `loadLegoData()` di dalam blok `init`. Fitur ini berguna ketika sudah cukup melihat isi suatu fungsi dan ingin kembali ke konteks pemanggil tanpa harus Step Over satu per satu hingga akhir fungsi.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

Jawaban:

Application class adalah class dasar yang disediakan oleh Android dan mewakili keseluruhan aplikasi itu sendiri. Ketika aplikasi pertama kali dijalankan, Android akan membuat instance dari class ini sebelum Activity, Fragment, atau komponen lain dibuat. Instance ini akan terus hidup selama aplikasi berjalan dan baru dihentikan ketika aplikasi ditutup.

Dalam arsitektur Android, Application class berada pada lapisan paling atas, Application class adalah “induk” dari semua komponen lain seperti Activity dan Fragment. Karena sifatnya yang hidup paling lama inilah, Application class menjadi tempat yang tepat untuk melakukan inisialisasi hal-hal yang dibutuhkan oleh seluruh aplikasi sejak awal.

Fungsi utama Application class:

- Inisialisasi library global, library seperti Timber (logging), Dagger/Hilt (dependency injection), atau Firebase perlu diinisialisasi sekali saja saat aplikasi mulai. Application class adalah tempat yang paling tepat untuk ini karena dipanggil sebelum komponen lain.
- Menyimpan state global, data atau objek yang perlu diakses dari mana saja dalam aplikasi bisa disimpan di sini, karena instance nya hanya satu dan selalu tersedia selama aplikasi hidup.
- Konfigurasi awal aplikasi, pengaturan seperti tema, bahasa default, ataupun konfigurasi jaringan bisa disiapkan disini.

TAUTAN GIT

<https://github.com/rikafaulianarahmi/MODUL-MOBILE/tree/main/MODUL%204>